

Parte 3 — Modelagem supervisionada e probabilidades

Tutorial completo do sciModelFlowR (versão 1.0.2)

Walter Esfrain Pereira

2026-09-19

Índice

1	Da pergunta ao modelo	3
2	Diagnóstico antes de acreditar	5
3	Complexidade justificada?	7
4	Probabilidades: classificação e a API de distribuições	9
5	Exercícios da Parte 3	14

1 Da pergunta ao modelo

O ensaio de campo simulado tem uma verdade declarada: o rendimento responde a nitrogênio (25 kg/ha por kg/ha), potássio (6), água (3) e matéria orgânica (120 por ponto percentual), além de efeitos de bloco e genótipo. Um analista que só dispõe das doses óbvias (nitrogênio e água) começa por um **modelo enxuto**; o modelo completo entra depois, com justificativa.

```
campo <- ler("field_yield")
v <- verdade$field_yield
sem_out <- campo[!campo$plot_id %in% v$outlier_ids, ]
cat("parcelas usadas:", nrow(sem_out), "(5 outliers excluídos)\n")
```

parcelas usadas: 175 (5 outliers excluídos)

```
fazer_reg <- function(preditores, categoricos = character(), familia = "linear_regression",
                      dados = sem_out) {
  spec <- smf_experiment_spec(
    task = smf_task_spec("regression", "yield"),
    data = smf_data_spec("yield", preditores, id_column = "plot_id",
                        categorical = categoricos),
    design = smf_design_spec(id_column = "plot_id"),
    preprocessing = smf_preprocess_spec("median"),
    resampling = smf_resampling_spec("holdout", train_prop = 0.7, seed = 21L),
    model = smf_model_spec(familia, "stats"),
    metrics = list(smf_metric_spec("rmse"), smf_metric_spec("rsq")),
    reproducibility = smf_reproducibility_spec(21L)
  )
  smf_fit_experiment(spec, dados)
}
enxuto <- fazer_reg(c("nitrogen", "water"))
completo <- fazer_reg(c("nitrogen", "potassium", "water", "organic_matter",
                      "block", "genotype"), categoricos = c("block", "genotype"))
tab_met <- data.frame(
  modelo = c("enxuto (N + água)", "completo (N + K + água + MO + bloco + genótipo)"),
  rmse = c(enxuto@metrics$value[1], completo@metrics$value[1]),
  rsq = c(enxuto@metrics$value[2], completo@metrics$value[2])
)
tab_smf(tab_met, caption = "Desempenho de validação: o modelo completo reduz o RMSE e explica
```

Tabela 1.1: Desempenho de validação: o modelo completo reduz o RMSE e explica mais variância.

modelo	rmse	rsq
enxuto (N + água)	494,071	0,836
completo (N + K + água + MO + bloco + genótipo)	463,714	0,855

```
coef_comp <- coef(smf_backend_object(completo))
tab_coef <- data.frame(
  parametro = c("nitrogen", "potassium", "water", "organic_matter"),
  verdade = c(v$beta[["nitrogen"]], v$beta[["potassium"]],
              v$beta[["water"]], v$beta[["organic_matter"]]),
  estimado = as.numeric(coef_comp[c("nitrogen", "potassium", "water", "organic_matter")])
)
tab_coef$erro <- tab_coef$estimado - tab_coef$verdade
tab_smf(tab_coef, caption = "Coeficientes do modelo completo contra a verdade plantada (efeitos de bloco e genótipo omitidos da tabela).")
```

Tabela 1.2: Coeficientes do modelo completo contra a verdade plantada (efeitos de bloco e genótipo omitidos da tabela).

parametro	verdade	estimado	erro
nitrogen	25	24,609	-0,391
potassium	6	8,139	2,139
water	3	2,953	-0,047
organic_matter	120	113,644	-6,356

i Interpretação

O modelo completo recupera os quatro coeficientes com erro de 6,4 kg/ha ou menos, e o RMSE de validação cai de 494 para 464. A diferença entre os dois modelos **não** é “complexidade”: é a estrutura correta dos dados. O pacote deixa claro o que entrou no ajuste: `smf_backend_object()` devolve o modelo nativo, e `fit@fit@predictors` lista os preditores usados.

2 Diagnóstico antes de acreditar

Um ajuste bom em média pode estar errado em regiões. `smf_diagnose()` verifica suposições e devolve resíduos, ajustados e a lista de checagens.

```
diag <- smf_diagnose(completo)
tab_diag <- data.frame(
  checagem = c("observações avaliadas", "todas as predições são finitas",
               "predições ausentes"),
  resultado = c(diag@checks$n, diag@checks$finite_predictions,
                diag@checks$missing_predictions)
)
tab_smf(tab_diag, caption = "Checagens de diagnóstico do modelo completo.")
```

Tabela 2.1: Checagens de diagnóstico do modelo completo.

checagem	resultado
observações avaliadas	53
todas as predições são finitas	1
predições ausentes	0

```
res_df <- data.frame(fitted = diag@fitted, resid = diag@residuals,
                    nitrogen = sem_out$nitrogen[completo@split@test_index])
p1 <- ggplot(res_df, aes(fitted, resid)) +
  geom_point(colour = cores_smf[["primaria"]], alpha = .7) +
  geom_hline(yintercept = 0, linetype = "dashed") +
  labs(x = "ajustado (kg/ha)", y = "resíduo (kg/ha)",
       title = "Residuals vs fitted")
p2 <- ggplot(res_df, aes(nitrogen, abs(resid))) +
  geom_point(colour = cores_smf[["destaque"]], alpha = .7) +
  geom_smooth(method = "loess", formula = y ~ x, se = FALSE,
             colour = cores_smf[["alerta"]]) +
  labs(x = "nitrogen (kg/ha)", y = "|resíduo| (kg/ha)",
       title = "|Residuals| vs nitrogen")
patchwork::wrap_plots(p1, p2)
```

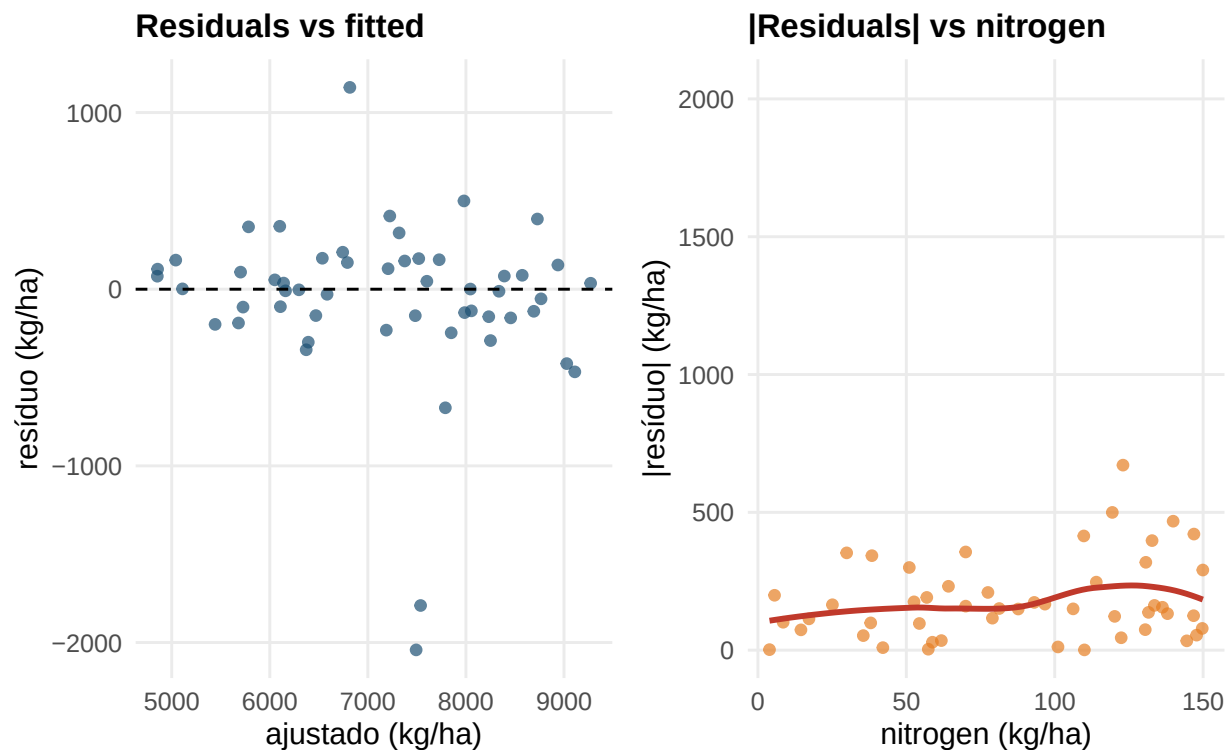


Figura 2.1: Resíduos contra ajustados e magnitude do resíduo contra nitrogênio: a dispersão cresce com a dose — a heterocedasticidade plantada aparece.

! Interpretação

Os resíduos centram em zero (sem viés grosseiro), mas a **magnitude cresce com a dose**: é exatamente o modelo plantado, $\sigma = 100 + 1,5 \times \text{nitrogen}$. Consequência prática: intervalos e testes que supõem variância constante serão otimistas nas doses altas. As Partes 5 e 6 tratam disso (bootstrap, conformal, modelos de incerteza); por ora, registre o diagnóstico.

3 Complexidade justificada?

Com o mesmo holdout e a mesma especificação, seis famílias de modelos. Se a flexibilidade não paga, a resposta é clara.

```
fazer_familia <- function(familia, engine) {
  spec <- smf_experiment_spec(
    task = smf_task_spec("regression", "yield"),
    data = smf_data_spec("yield", c("nitrogen", "potassium", "water", "organic_matter",
                                     "block", "genotype"),
                        id_column = "plot_id", categorical = c("block", "genotype")),
    design = smf_design_spec(id_column = "plot_id"),
    preprocessing = smf_preprocess_spec("median"),
    resampling = smf_resampling_spec("holdout", train_prop = 0.7, seed = 21L),
    model = smf_model_spec(familia, engine),
    metrics = list(smf_metric_spec("rmse"), smf_metric_spec("rsq")),
    reproducibility = smf_reproducibility_spec(21L)
  )
  smf_fit_experiment(spec, sem_out)
}

familias <- data.frame(
  familia = c("linear_regression", "decision_tree", "random_forest",
              "boosted_tree", "svm_rbf", "nearest_neighbor"),
  engine = c("stats", "tidymodels", "tidymodels", "tidymodels", "tidymodels", "tidymodels")
)

tab_fam <- do.call(rbind, lapply(seq_len(nrow(familias)), function(i) {
  f <- fazer_familia(familias$familia[i], familias$engine[i])
  data.frame(familia = familias$familia[i],
             rmse = f@metrics$value[f@metrics$metric == "rmse"],
             rsq = f@metrics$value[f@metrics$metric == "rsq"])
}))

tab_fam <- tab_fam[order(tab_fam$rmse), ]
tab_smf(tab_fam, caption = "Seis famílias no mesmo holdout: RMSE e R2 de validação.")
```

Tabela 3.1: Seis famílias no mesmo holdout: RMSE e R² de validação.

	familia	rmse	rsq
1	linear_regression	463,714	0,855
5	svm_rbf	520,777	0,818
2	decision_tree	547,112	0,799
4	boosted_tree	590,740	0,765

	familia	rmse	rsq
3	random_forest	624,697	0,738
6	nearest_neighbor	1066,772	0,235

```
ggplot(tab_fam, aes(stats::reorder(familia, -rmse), rmse)) +
  geom_col(fill = cores_smf[["primaria"]], width = .6) +
  geom_text(aes(label = fmt(rmse, 0)), vjust = -0.4) +
  labs(x = NULL, y = "RMSE de validação (kg/ha)",
       title = "Model families on the same holdout") +
  ylim(0, max(tab_fam$rmse) * 1.12)
```

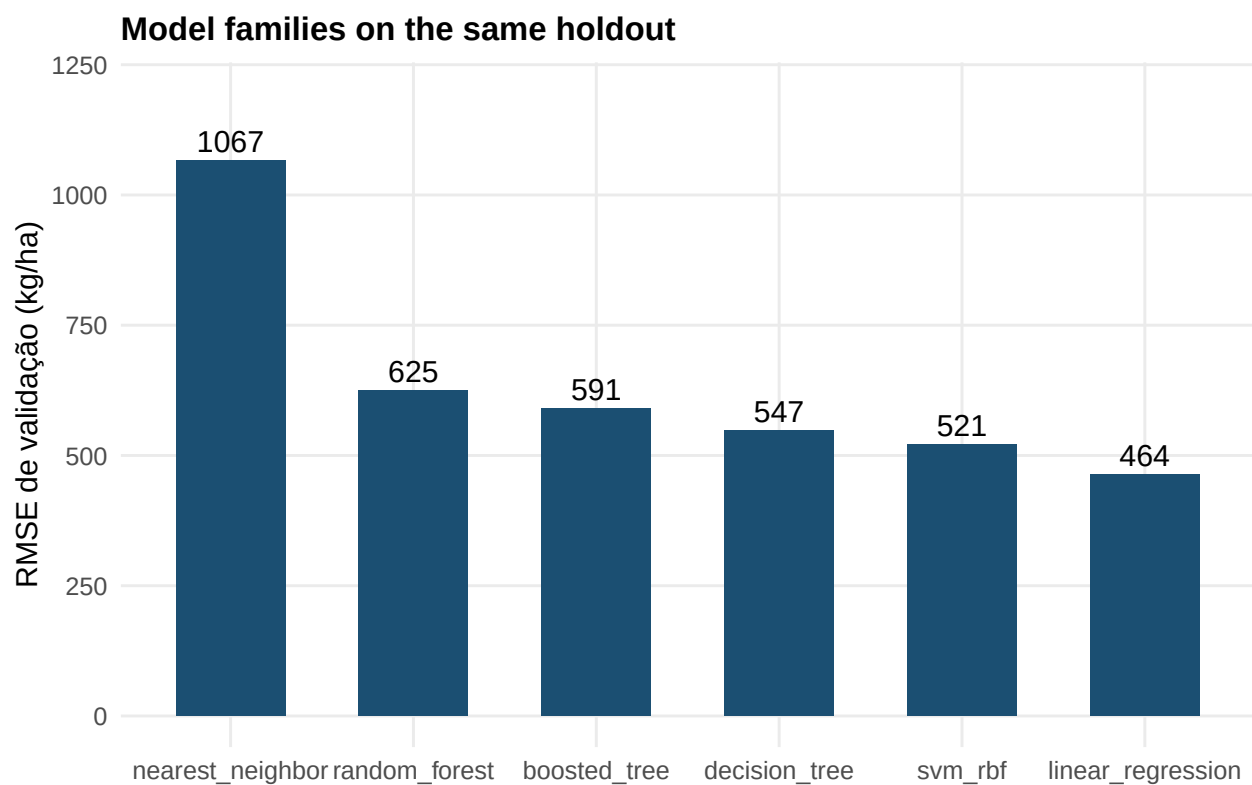


Figura 3.1: Comparação de famílias: a reta recupera a verdade; árvores e vizinhos mais próximos não superam a estrutura linear correta.

⚠ Armadilha

A tentação é sempre “usar o modelo mais forte”. Aqui, o modelo **linear** vence porque os dados foram gerados lineares — e nenhuma família flexível compensa a variância extra. Em outros conjuntos a resposta se inverte (veremos na classificação, logo abaixo). O critério não é o prestígio do método: é o desempenho em observações novas, medido no desenho certo.

4 Probabilidades: classificação e a API de distribuições

No conjunto de doença, o desfecho é binário, desbalanceado e com 5% de rótulos invertidos — e a fronteira verdadeira tem um termo quadrático. Compare a regressão logística com uma floresta aleatória.

```
doenca <- ler("disease_binary")
doenca$disease <- factor(doenca$disease, levels = c(0, 1))
fazer_cls <- function(familia, engine) {
  spec <- smf_experiment_spec(
    task = smf_task_spec("binary", "disease", positive_label = "1"),
    data = smf_data_spec("disease", c("x1", "x2", "x3", "x4"), id_column = "plant_id"),
    design = smf_design_spec(id_column = "plant_id"),
    preprocessing = smf_preprocess_spec("median", center = TRUE, scale = TRUE),
    resampling = smf_resampling_spec("stratified_holdout", train_prop = 0.75,
                                     strata = "disease", seed = 22L),
    model = smf_model_spec(familia, engine),
    metrics = list(smf_metric_spec("log_loss"), smf_metric_spec("brier"),
                  smf_metric_spec("roc_auc")),
    reproducibility = smf_reproducibility_spec(22L)
  )
  smf_fit_experiment(spec, doenca)
}
logit <- fazer_cls("logistic_regression", "stats")
floresta <- fazer_cls("random_forest", "tidymodels")
tab_cls <- rbind(
  cbind(modelo = "logística", logit@metrics[, c("metric", "value")]),
  cbind(modelo = "floresta aleatória", floresta@metrics[, c("metric", "value")])
)
tab_smf(tab_cls, caption = "Classificação da doença: a floresta captura o termo quadrático qu
```

Tabela 4.1: Classificação da doença: a floresta captura o termo quadrático que a logística não vê.

modelo	metric	value
logística	log_loss	0,379
logística	brier	0,221
logística	roc_auc	0,652
floresta aleatória	log_loss	0,610
floresta aleatória	brier	0,180
floresta aleatória	roc_auc	0,804

modelo	metric	value
--------	--------	-------

```

teste <- doenca[floresta@split@test_index, ]
p_rf <- floresta@prediction@probabilities[, "1"]
p_lg <- logit@prediction@probabilities[, "1"]
curva <- function(p, y) {
  o <- order(p, decreasing = TRUE)
  y <- as.integer(y[o] == "1")
  data.frame(fpr = cumsum(1 - y) / sum(1 - y),
             tpr = cumsum(y) / sum(y))
}
roc <- rbind(cbind(modelo = "logística", curva(p_lg, teste$disease)),
            cbind(modelo = "floresta", curva(p_rf, teste$disease)))

```

```

ggplot(roc, aes(fpr, tpr, colour = modelo)) +
  geom_line(linewidth = 1) +
  geom_abline(slope = 1, intercept = 0, linetype = "dashed", colour = "grey60") +
  scale_colour_manual(values = c(logística = cores_smf[["neutro"]],
                                floresta = cores_smf[["primaria"]])) +
  coord_equal() +
  labs(x = "falsos positivos (1 - especificidade)", y = "verdadeiros positivos (sensibilidade)",
       colour = NULL,
       title = "ROC curves on the test partition",
       subtitle = paste0("AUC logística = ",
                         fmt(logit@metrics$value[logit@metrics$metric == "roc_auc"], 3),
                         " | floresta = ",
                         fmt(floresta@metrics$value[floresta@metrics$metric == "roc_auc"], 3),
                         " | teto oráculo = 0,84")) +
  theme(legend.position = "top")

```

ROC curves on the test partition

AUC logística = 0,652 | floresta = 0,804 | teto oráculo = 0,84

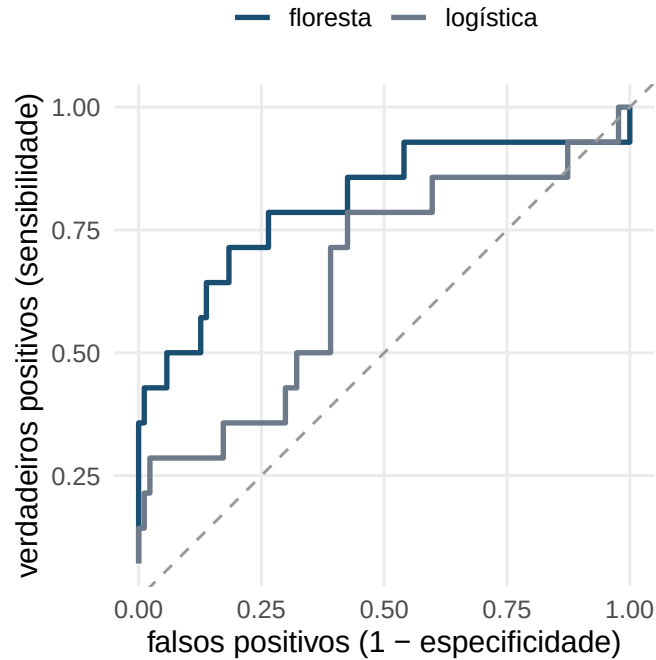


Figura 4.1: Curvas ROC na partição de teste: a floresta se aproxima do teto imposto pelo ruído de rótulo (AUC oráculo = 0,84).

A predição probabilística é um objeto de distribuição. O pacote tem uma API própria para operar essas distribuições — média, quantis, log-probabilidade, intervalos — e para avaliá-las com escores próprios.

```
pd_rf <- smf_prediction_distribution(  
  "class_probabilities",  
  list(prob = floresta@prediction@probabilities, classes = c("0", "1"))  
)  
escores_rf <- smf_evaluate_probabilistic(teste$disease, pd_rf,  
                                         c("log_loss", "brier", "ece"))  
tab_smf(escores_rf, caption = "Escores probabilísticos da floresta (ECE: erro de calibração e
```

Tabela 4.2: Escores probabilísticos da floresta (ECE: erro de calibração esperado).

metric	value	direction	n
log_loss	0,610	minimize	101
brier	0,180	minimize	101
ece	0,081	minimize	101

```

normal <- smf_prediction_distribution(
  "parametric",
  list(distribution = "normal", mean = c(5000, 6000), sd = c(300, 450))
)
tab_dist <- data.frame(
  operacao = c("média", "quantil 5%", "quantil 95%", "log-probabilidade em 5000"),
  resultado = c(paste(fmt(smf_dist_mean(normal), 0), collapse = " | "),
    paste(fmt(smf_dist_quantile(normal, 0.05), 0), collapse = " | "),
    paste(fmt(smf_dist_quantile(normal, 0.95), 0), collapse = " | "),
    paste(fmt(smf_dist_log_prob(normal, c(5000, 5000)), 3), collapse = " | "))
)
tab_smf(tab_dist, caption = "Operações sobre distribuições preditivas: duas parcelas hipotéticas")

```

Tabela 4.3: Operações sobre distribuições preditivas: duas parcelas hipotéticas com médias 5000 e 6000 kg/ha.

operacao	resultado
média	5000 6000
quantil 5%	4507 5260
quantil 95%	5493 6740
log-probabilidade em 5000	-6,623 -9,497

```

prob_df <- data.frame(p = p_rf, classe = teste$disease)
ggplot(prob_df, aes(p, fill = classe)) +
  geom_histogram(position = "identity", alpha = .55, bins = 25, colour = "white") +
  scale_fill_manual(values = c(`0` = cores_smf[["claro"]], `1` = cores_smf[["alerta"]]),
    labels = c("sadia (0)", "doente (1)")) +
  labs(x = "probabilidade prevista de doença", y = "plantas", fill = NULL,
    title = "Predicted probabilities by true class") +
  theme(legend.position = "top")

```

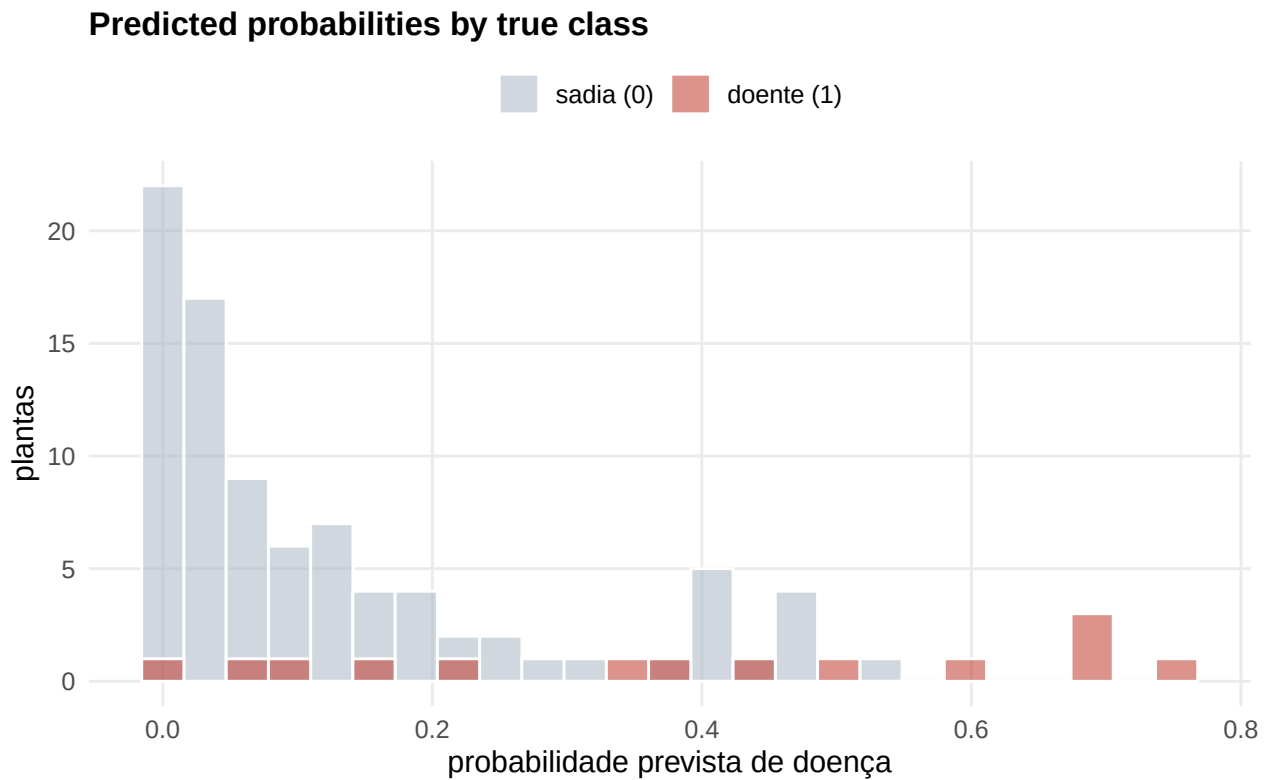


Figura 4.2: Distribuição das probabilidades previstas por classe verdadeira: a sobreposição é o ruído de rótulo em ação.

i Interpretação

A floresta quase dobra a separação da logística (AUC 0,80 contra 0,65), mas nenhum modelo pode chegar a 1,0: o ruído de rótulo impõe um teto (AUC oráculo = 0,84) e as distribuições de probabilidade se sobrepõem. Um classificador honesto comunica **probabilidade**, não certeza — e a calibração dessas probabilidades é o tema da Parte 4.

5 Exercícios da Parte 3

Exercício 3.1 — O preço dos cinco pontos

Ref faça o modelo completo **com** os cinco outliers e compare RMSE, R^2 e o coeficiente da matéria orgânica com os do modelo sem outliers. Quantos pontos mudaram a conclusão?

Exercício 3.2 — Dar olhos à logística

Crie a coluna $x3_sq = x3^2$ no conjunto de doença e repita a logística com $x1$, $x2$, $x3$, $x3_sq$, $x4$. O AUC se aproxima do oráculo (0,84)? O que isso diz sobre “modelo simples contra complexo”?

Exercício 3.3 — Acurácia engana

Com limiar 0,5, calcule a acurácia de um classificador que prevê “sadia” para todas as plantas. Compare com a acurácia da floresta no teste e explique a diferença em termos de prevalência.

Exercício 3.4 — Varredura de limiar

Para a floresta, calcule sensibilidade e especificidade em limiares de 0,1 a 0,9 e descreva o compromisso. Onde você colocaria o limiar para um programa de controle de doença em que um falso negativo custa dez vezes mais que um falso positivo?

Soluções

3.1. Com os outliers, o RMSE sobe, o R^2 cai e o coeficiente da matéria orgânica se afasta da verdade (como na Parte 2, quase triplica). Cinco pontos em 180 bastam para mudar a conclusão — influência desproporcional.

3.2. Com $x3_sq$, o AUC da logística sobe substancialmente (o termo quadrático era o que faltava). O mesmo modelo “simples” passa a competir: o problema nunca foi a família, foi a **forma funcional**.

3.3. Prever “sadia” para todas acerta a fração de negativas (cerca de 90%), mas tem sensibilidade zero. A floresta tem acurácia menor que isso em muitos limiares e ainda assim é infinitamente mais útil: acurácia não mede o que interessa com desbalanceamento.

3.4. Sensibilidade cresce e especificidade cai conforme o limiar diminui. Com falso negativo dez vezes mais caro, o limiar deve ser **baixo** (mais plantas classificadas como doentes), aceitando mais falsos positivos. A Parte 4 formaliza isso com custos explícitos.